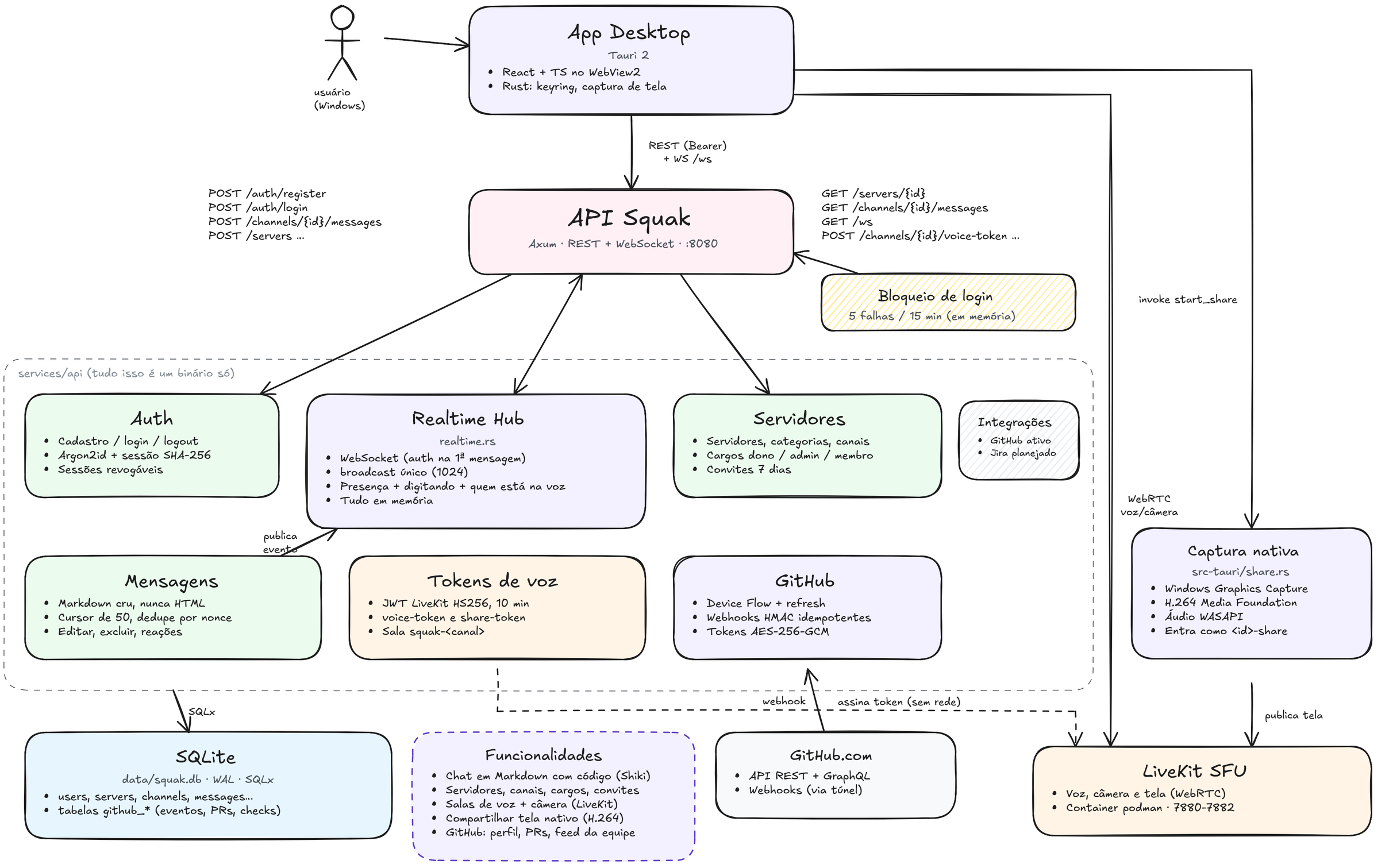
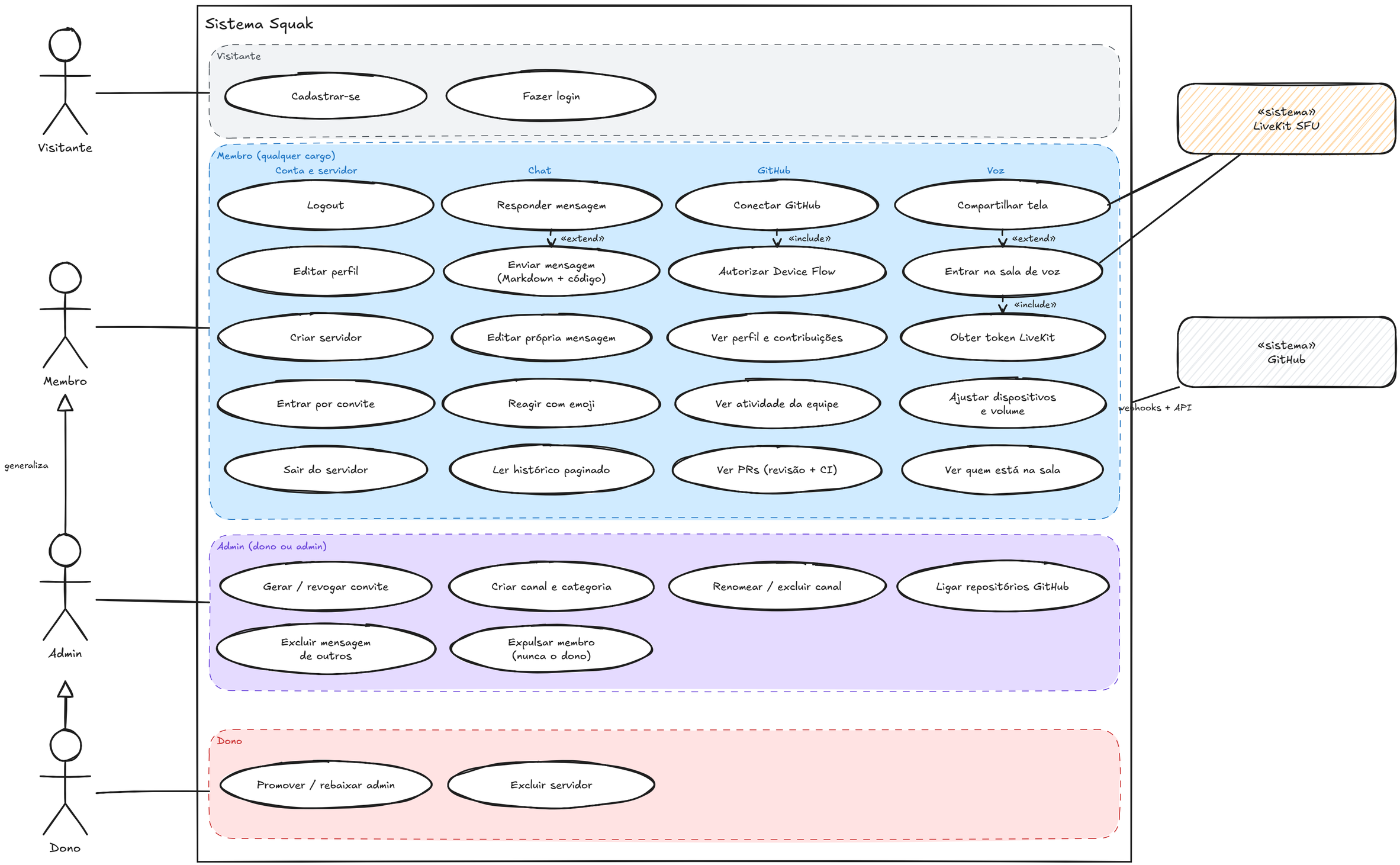
squak, como tá hoje

um binário Rust com SQLite. áudio e vídeo vão direto pro LiveKit, a API só assina o token.



Casos de uso

Associação de um ator vale para todos os casos do grupo. Admin herda Membro; Dono herda Admin.



Modelo entidade-relacionamento

Migrations 0001_init.sql e 0002_github.sql. Seta = chave estrangeira (N → 1). Tracejado = relação lógica sem FK.

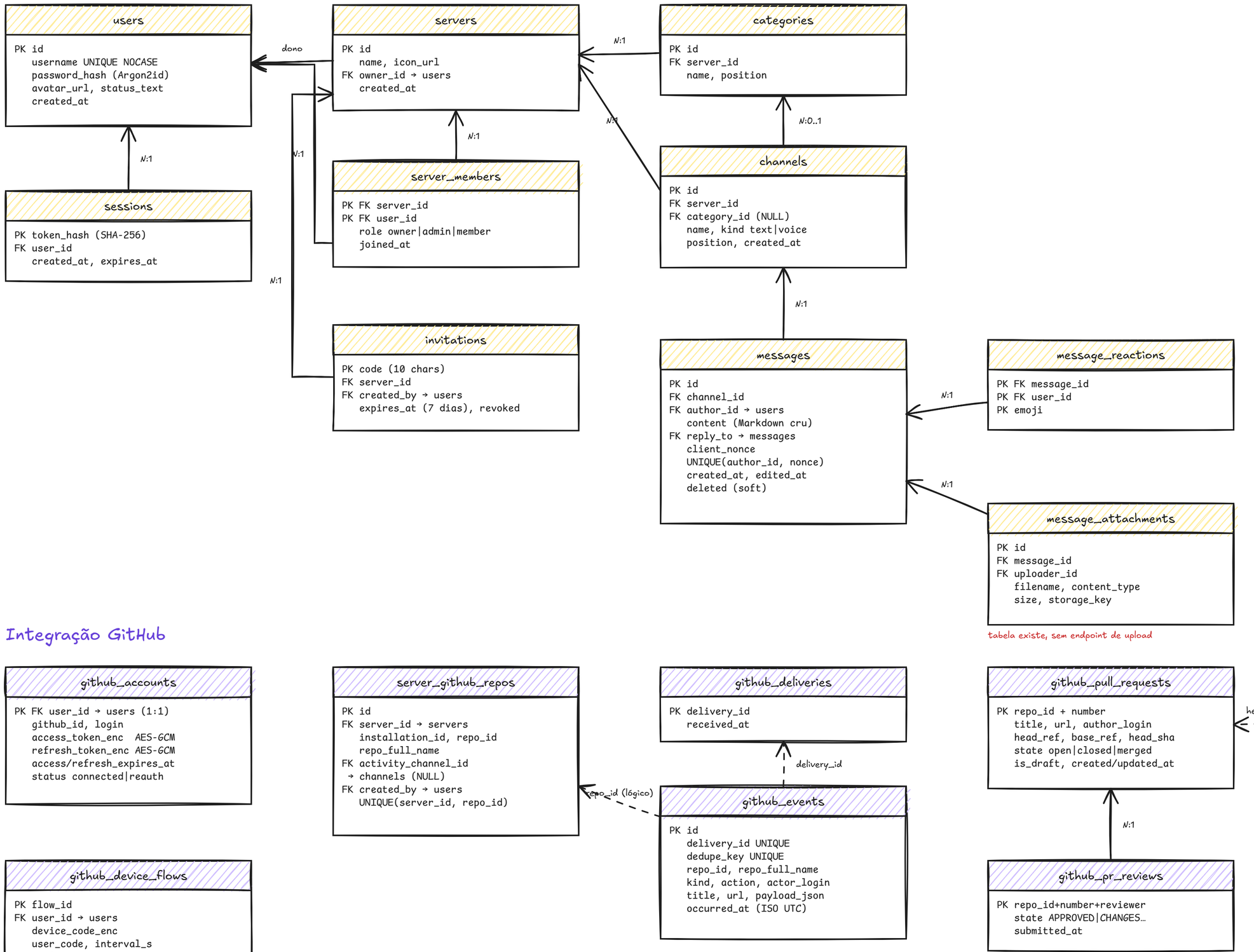
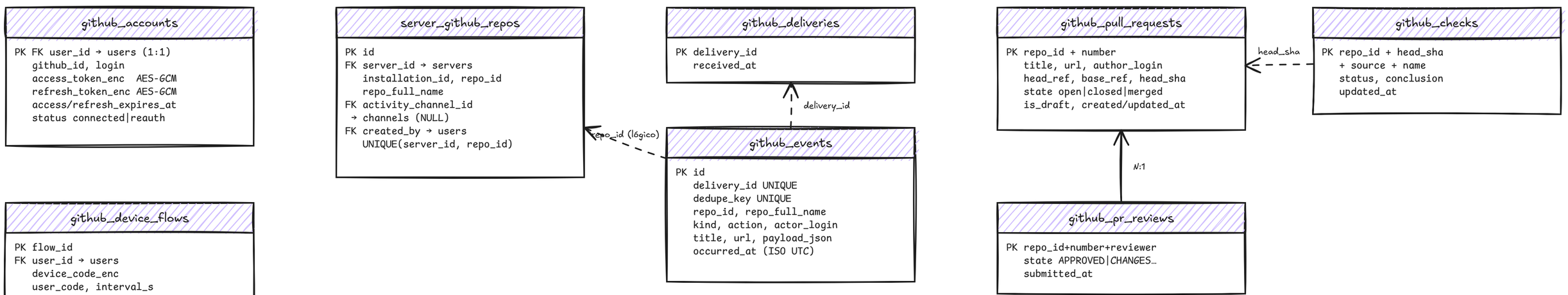


tabela existe, sem endpoint de upload

Integração GitHub

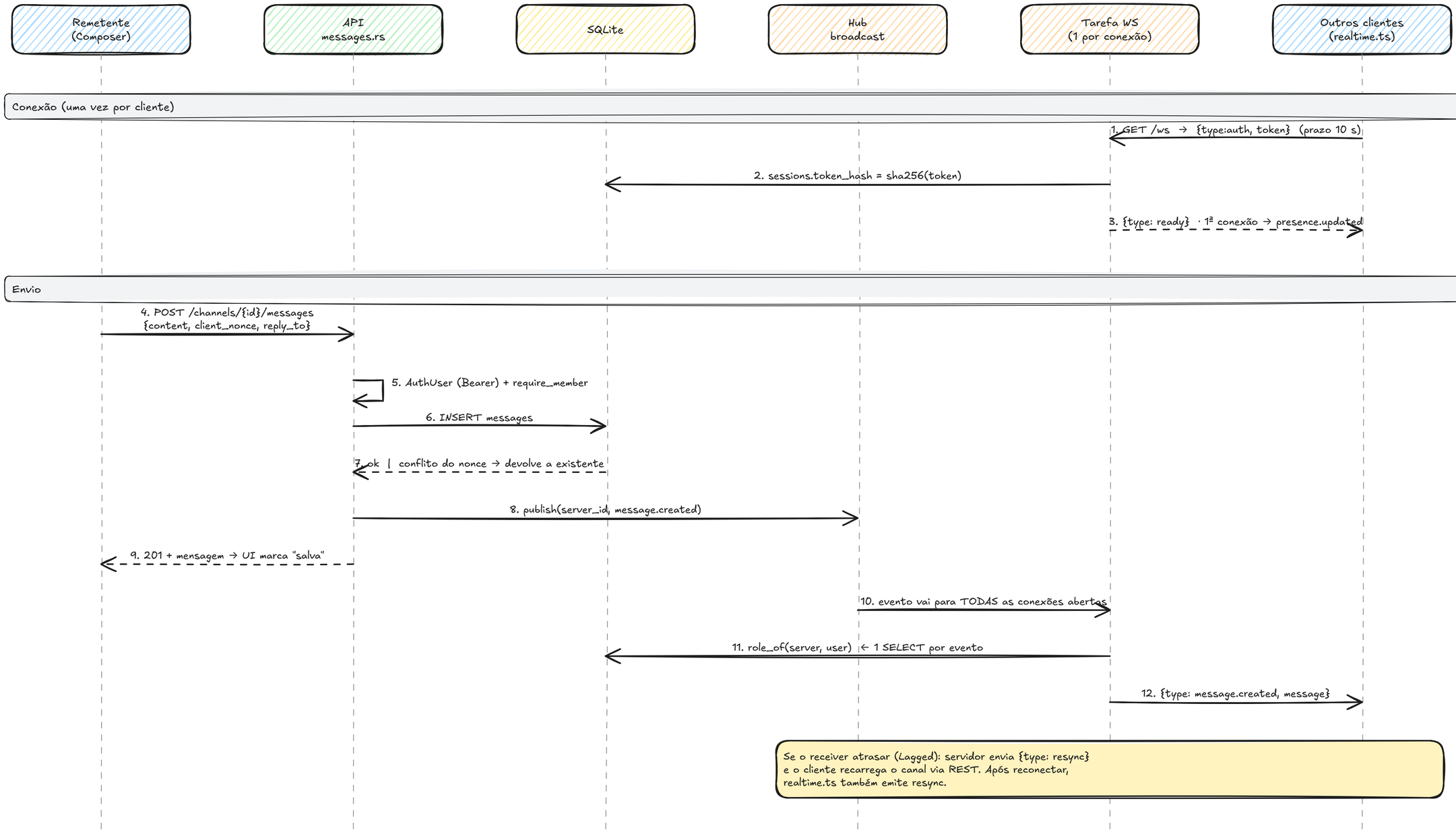


Observações

- FKs entre seções aparecem como → na própria coluna.
- github_events / PRs / checks não têm FK para servers: o fan-out para workspaces é feito por server_github_repos.repo_id no momento do webhook.
- Mensagem apagada: deleted = 1 e content zerado. Autor, data e reações ficam.
- Sessões guardam só o hash do token; o token cru fica no keyring do Windows.
- Tokens GitHub cifrados; a chave (GITHUB_TOKEN_KEY) nunca vai ao banco.

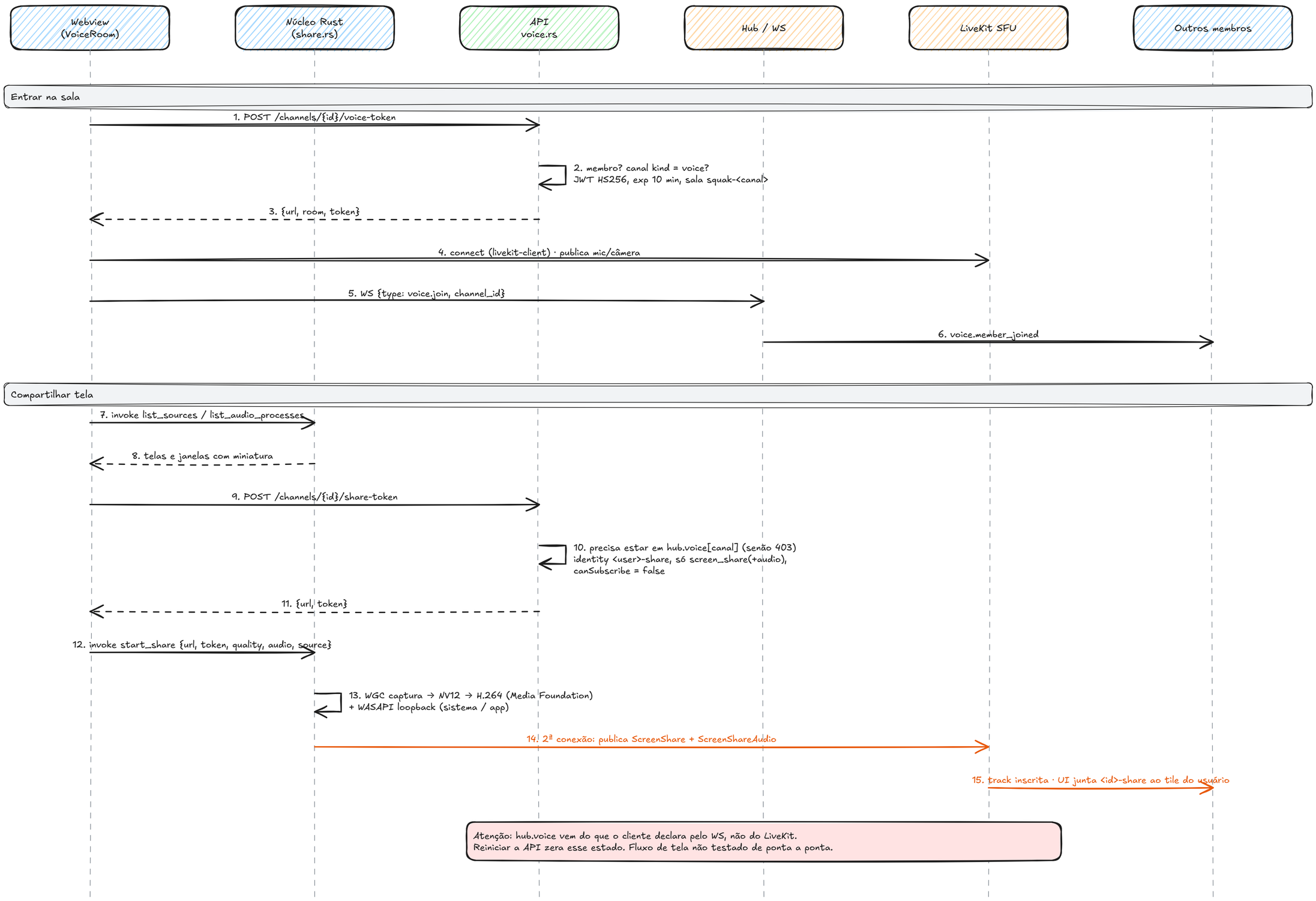
Enviar mensagem em tempo real

REST grava, WebSocket só notifica. A UI marca como salva só depois da resposta da API.



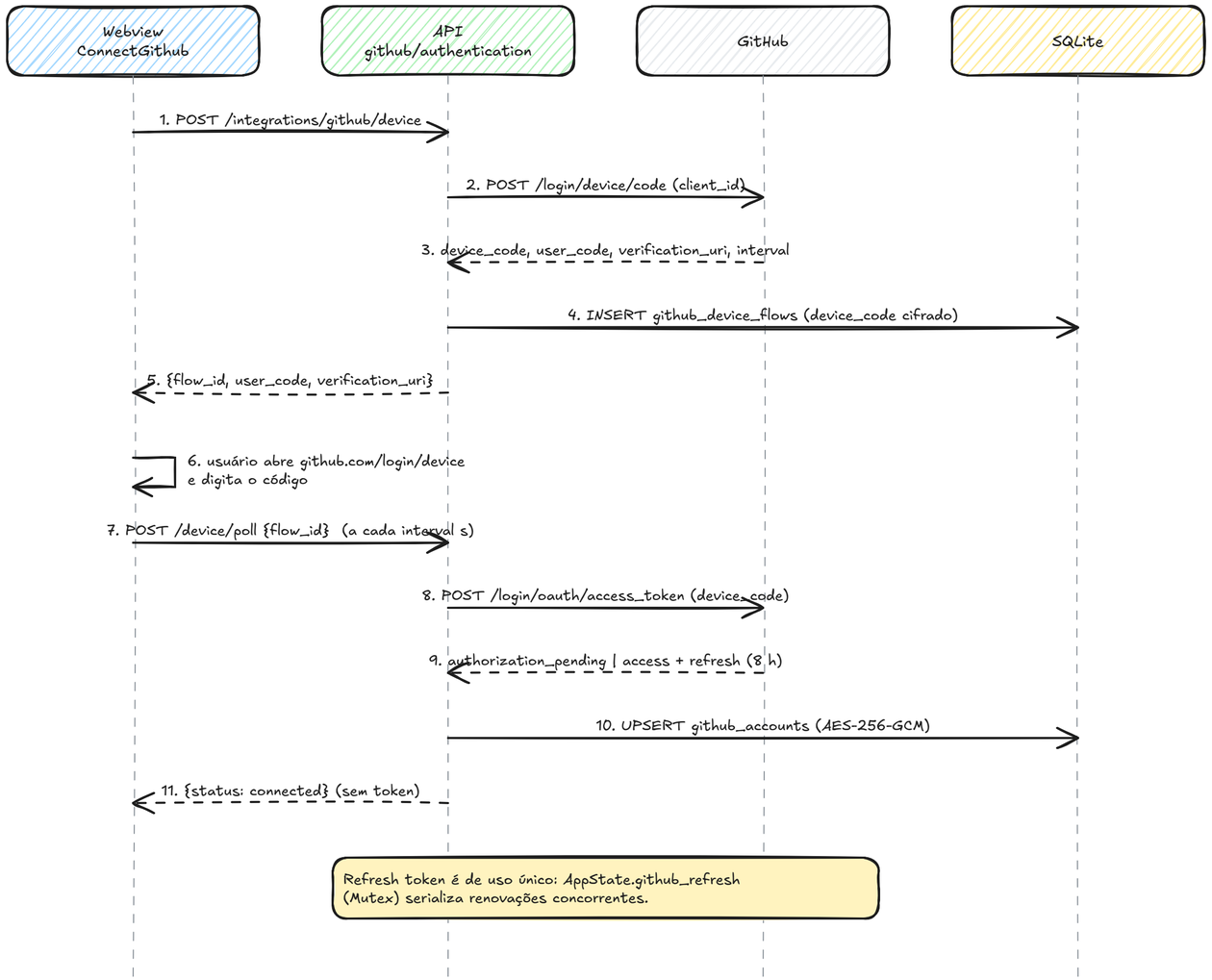
Entrar na voz e compartilhar a tela

Mídia nunca passa pela API. Duas conexões no LiveKit por usuário quando transmite: webview (voz) e Rust (tela).



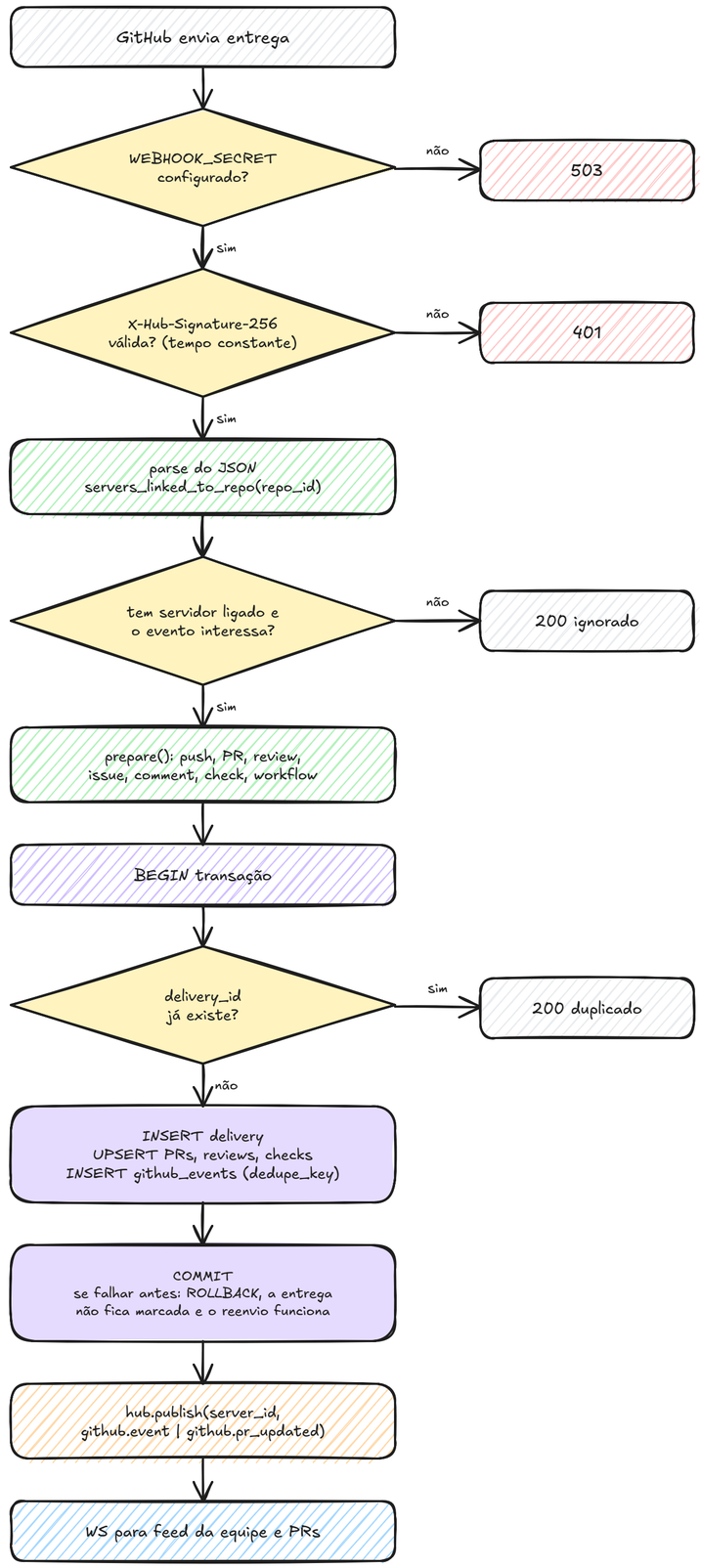
Conectar GitHub (Device Flow)

Login no Squak e conta GitHub são independentes. O app nunca vê token do GitHub.



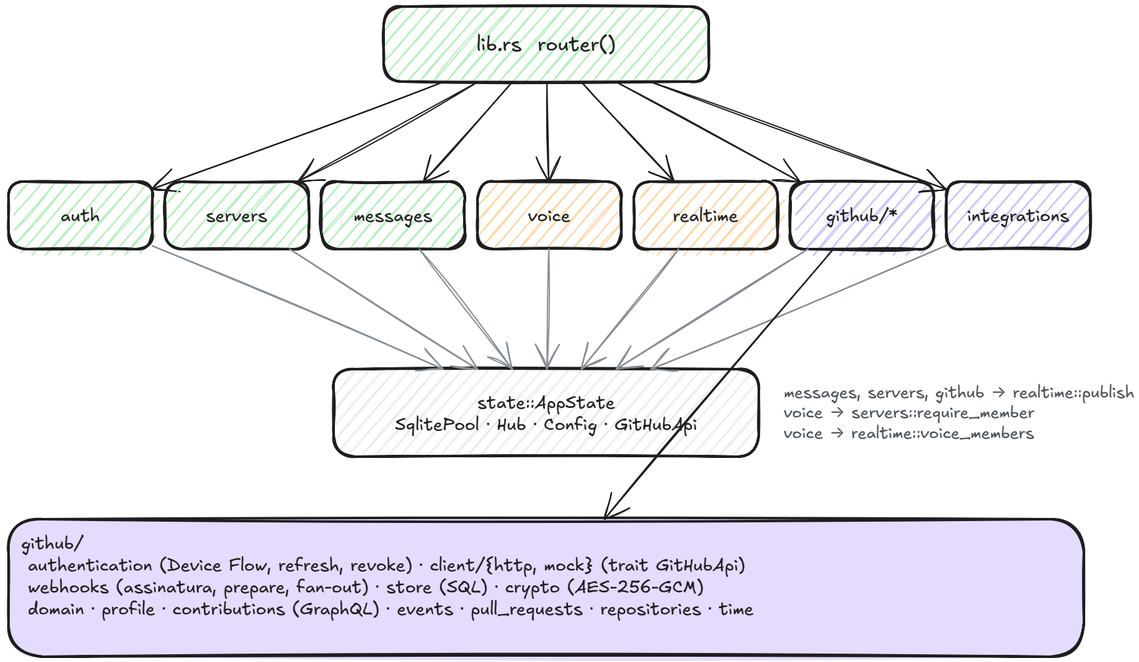
Pipeline do webhook

POST /webhooks/github (rota pública, corpo até 8 MB)

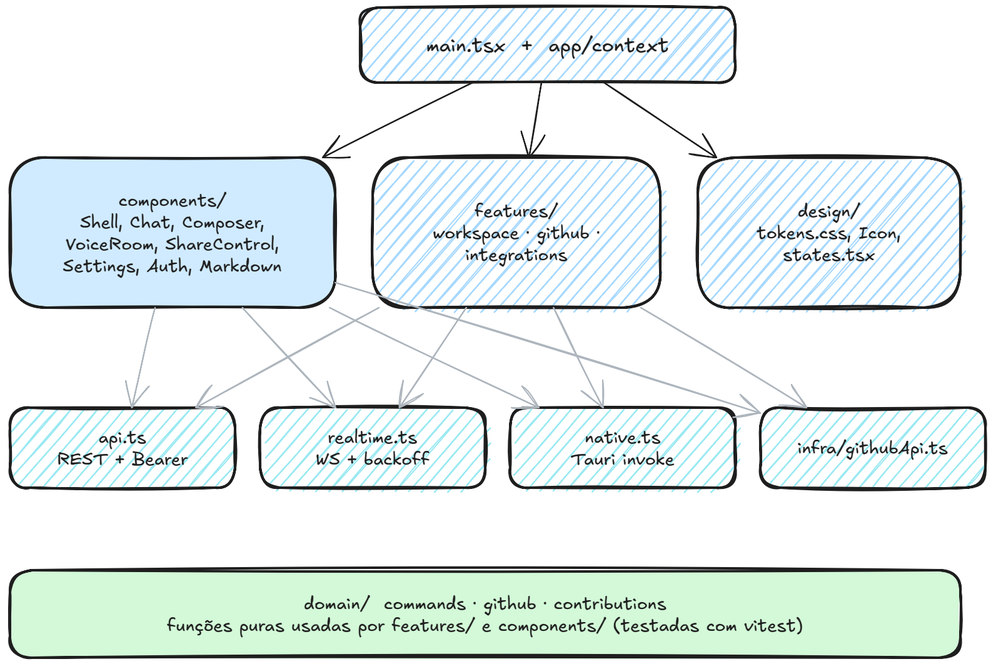


Componentes (dependências de código)

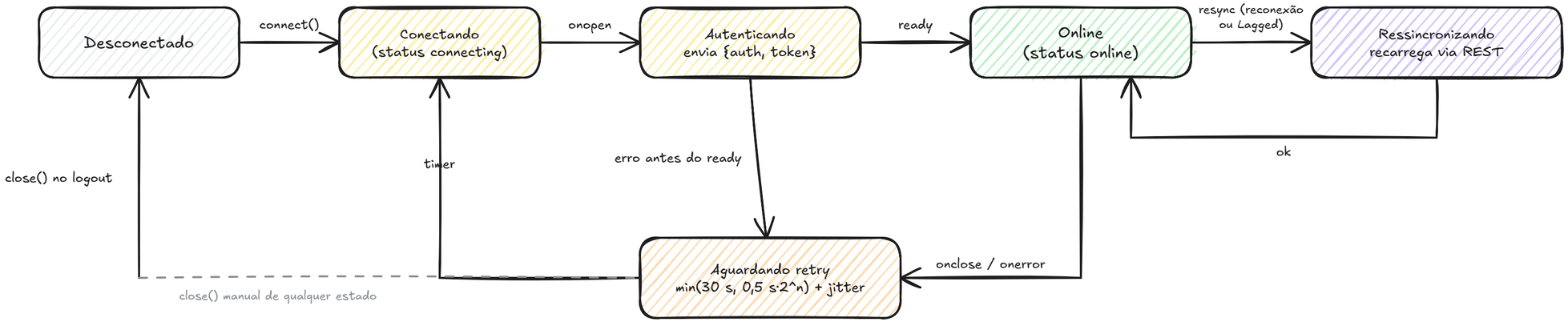
backend (services/api/src)



frontend (apps/desktop/src)



Máquina de estados do cliente WebSocket (realtime.ts)



Implantação atual (dev local) e riscos

O que existe hoje roda inteiro numa máquina Windows. A coluna da direita é proposta, não implementado.

Máquina Windows (dev)

Tauri (janela nativa) ou
Vite dev server :1420 no navegador
(sessão em localStorage no modo navegador)

squak-api (cargo run)
127.0.0.1:8080
CORS: qualquer origem

webhooks

Túnel
cloudflared / ngrok

podman machine (VM Linux)

container livekit/livekit-server
portas 7880, 7881, 7882/udp
chaves do .env (scripts/livekit-up.ps1)

data/squak.db (arquivo)
data/uploads (configurado, sem uso)

Build: .cargo/config.toml manda o target para C:/pt-target (libwebrtc estoura o limite de 260 caracteres de caminho).
Webhooks GitHub exigem URL pública: túnel (cloudflared / ngrok) até :8080.
Sem credenciais da GitHub App: status unconfigured, ou GITHUB_MOCK=1 com dados simulados.

Proposta para sair do localhost (não implementado)

- Proxy reverso com TLS (Caddy / nginx) na frente da API; CORS restrito
- Hub com pub/sub externo (Redis ou NATS) para rodar várias instâncias
- Postgres no lugar do SQLite quando houver escrita concorrente de verdade
- LiveKit com TURN (ou LiveKit Cloud) para quem está atrás de NAT
- Webhooks do LiveKit (participant_joined/left) como fonte da verdade da voz
- Storage de objetos (S3 / R2) para anexos
- Instalador Windows assinado + auto-update do Tauri

riscos que achei lendo o código

Fan-out caro
Cada evento vai para TODAS as conexões e cada uma roda role_of() no SQLite.
500 conexões = 500 SELECTs por mensagem. Cachear membership por conexão.

Broadcast único (1024)
Um canal global para todos os servidores: cliente lento recebe Lagged e resync
à toa. Um canal por servidor reduz ruído e trabalho.

Estado em memória
Presença, membros de voz e tentativas de login somem ao reiniciar a API
e impedem escalar horizontalmente.

Voz declarada pelo cliente
voice.join só confere se é membro; não confirma que entrou no LiveKit.
O share-token depende desse estado.

Token LiveKit de 10 min
Confirme se a VoiceRoom pede token novo ao reconectar depois de 10 min;
senão a reconexão falha.

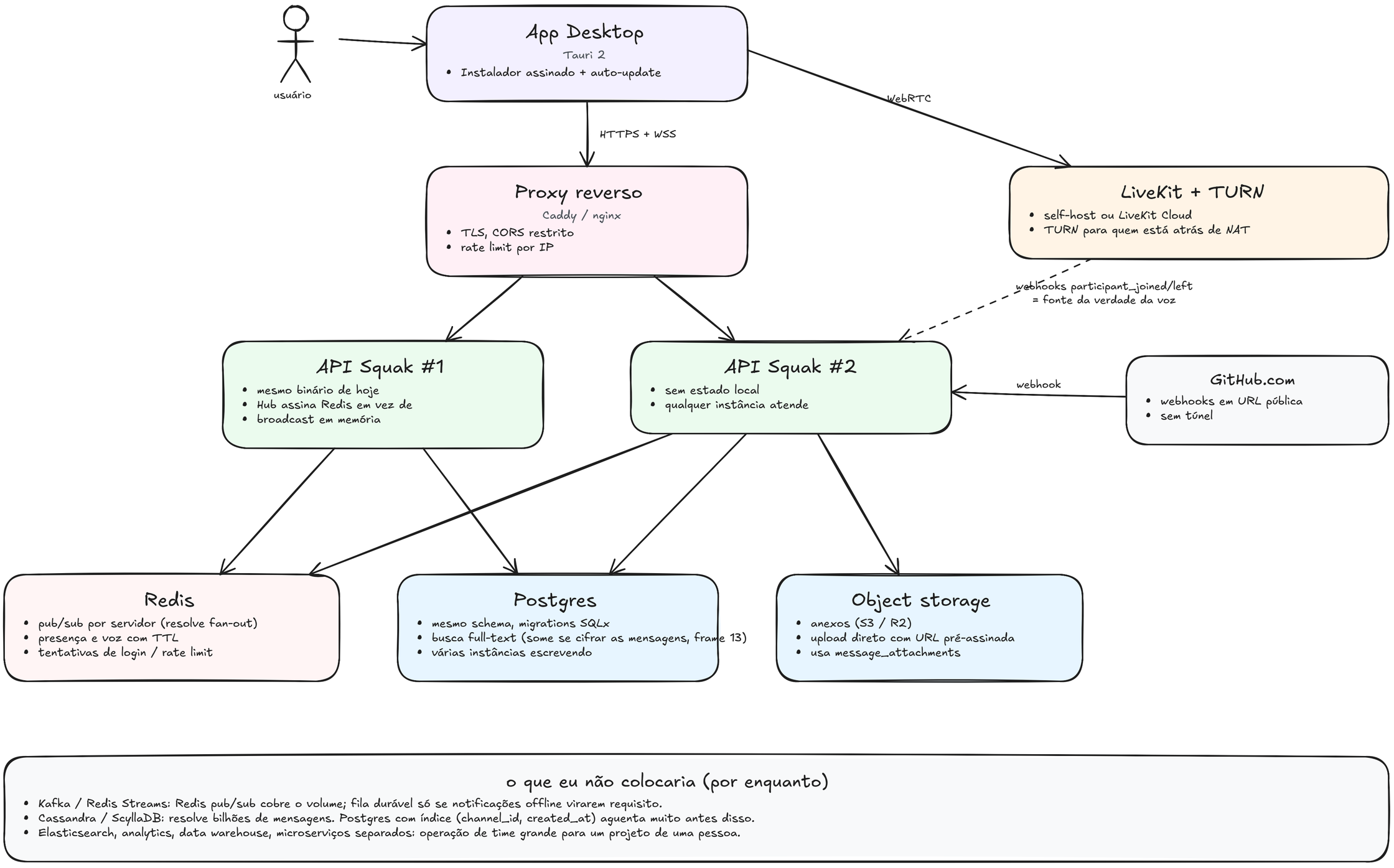
Cobertura de testes
WebSocket, mídia e compartilhamento de tela sem teste automatizado;
GitHub só testado com mock.

Schema sem uso
message_attachments e UPLOAD_DIR existem sem endpoint: ou implementa, ou tira.

CORS Any
Seguro só porque a auth é Bearer e o bind é 127.0.0.1. Fecha antes de expor.

se um dia sair do localhost

ainda um binário só. cada peça nova resolve um dos riscos do frame 8.



o que eu não colocaria (por enquanto)

- Kafka / Redis Streams: Redis pub/sub cobre o volume; fila durável só se notificações offline virarem requisito.
- Cassandra / ScyllaDB: resolve bilhões de mensagens. Postgres com índice (channel_id, created_at) aguenta muito antes disso.
- Elasticsearch, analytics, data warehouse, microserviços separados: operação de time grande para um projeto de uma pessoa.

design patterns que aparecem no código

o arquivo tá entre parênteses em cada post-it

backend

MONÓLITO MODULAR
um módulo por domínio, um router só em lib.rs, sobe um binário (services/api/src)

STRATEGY
trait GitHubApi com HttpGitHub e MockGitHub. GITHUB_MOCK escolhe (github/client/mod.rs)

INJEÇÃO DE DEPENDÊNCIA
AppState chega nos handlers por State<>; no teste entra o mock (state.rs)

OBSERVER / PUB-SUB
Hub.publish(server, evento) e cada conexão WS assina (realtime.rs)

PLUGIN REGISTRY
trait IntegrationProvider. Jira já registrado como planned (integrations.rs)

REPOSITORY (SÓ NO GITHUB)
store.rs junta o SQL do GitHub. servers e messages fazem SQL no próprio handler

RESULT + ERRO TIPADO
ApiError vira status HTTP, GhError chega classificado (error.rs)

isso aqui tá inconsistente, escolhe um jeito só

segurança e falhas

GUARD NO EXTRACTOR
AuthUser valida o Bearer antes do handler rodar (auth.rs)

CAPABILITY TOKEN
JWT do LiveKit com o mínimo: o -share só publica tela, 10 min (voice.rs)

IDEMPOTENCY KEY
client_nonce na mensagem, X-GitHub-Delivery e dedupe_key no webhook

BACKOFF + JITTER
reconecta em min(30s, 0,5s·2^n) + até 300ms aleatório (realtime.ts)

front

FACADE
githubApi.ts: componente nunca monta URL

FUNCTIONAL CORE
domain/*.ts é função pura, testada no vitest

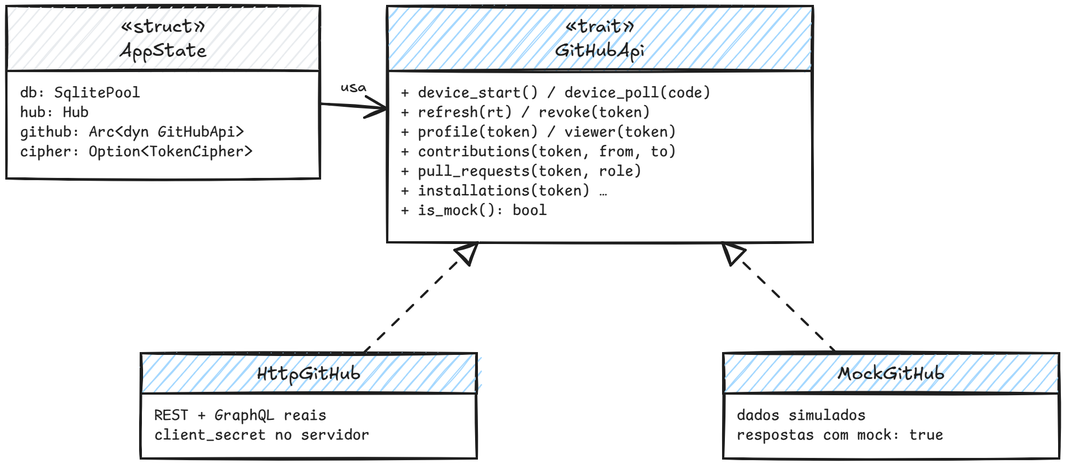
ESTADO DISCRIMINADO
loading | ready | failed; seq descarta resposta atrasada (useResource.ts)

CONTEXT
AppContext com me, rt, workspace. useApp() quebra sem provider

os patterns principais em diagrama de classe

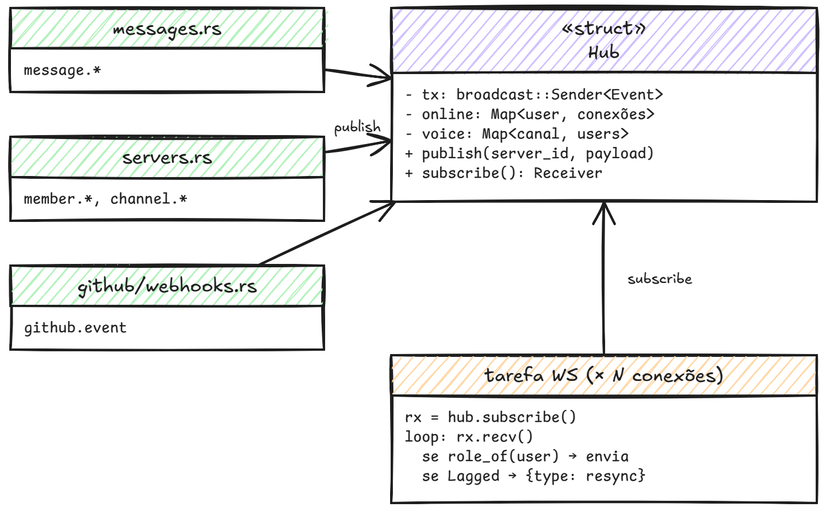
tracejado com triângulo: implementa. seta cheia: usa ou chama.

A · Strategy / Ports & Adapters (GitHub)



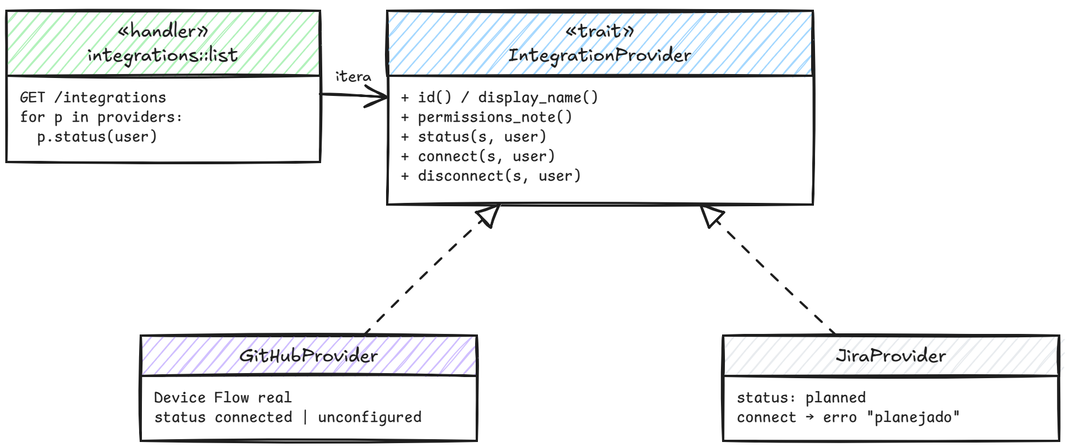
GITHUB_MOCK decide qual entra.
nos testes o mock vem por with_github().

B · Observer / Pub-Sub (tempo real)



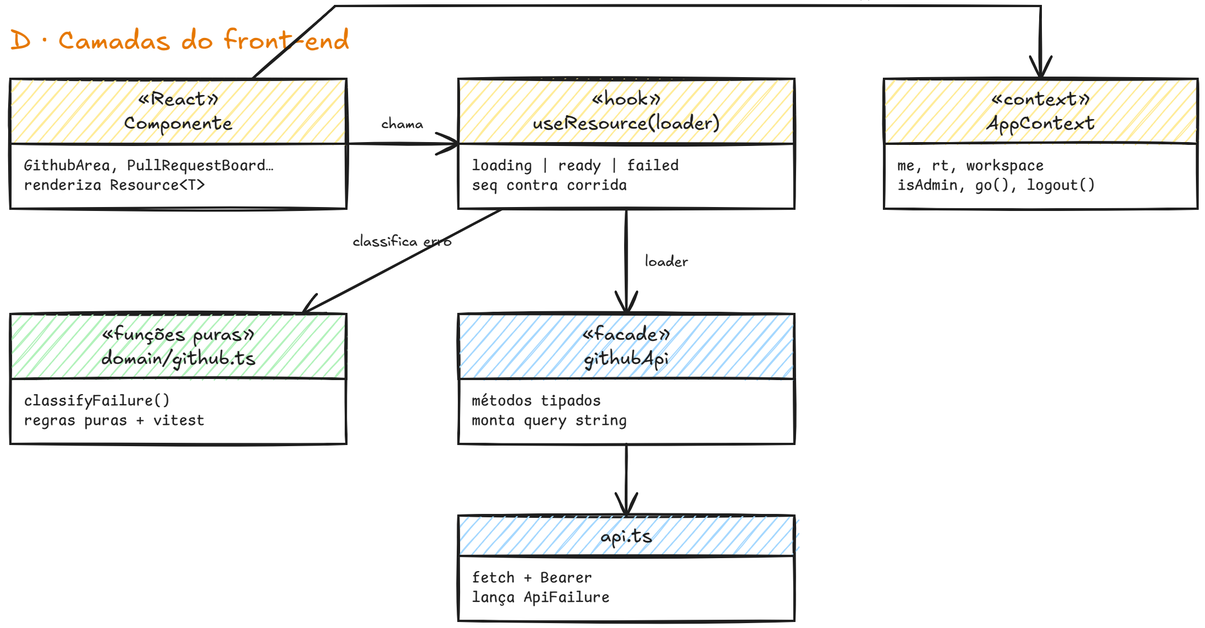
problema: um broadcast só pra todos os servidores,
toda conexão recebe tudo e vai no banco pra filtrar.

C · Plugin Registry (integrações)



provedor novo: uma struct e uma linha na lista.

D · Camadas do front-end



coisas pra arrumar

- Hub.publish_channel recebe channel_id e ignora (parâmetro morto): ou filtra por canal, ou remove.
- github/ tem store.rs, mas servers.rs e messages.rs fazem SQL dentro do handler. Padronize um lado (store por domínio).

o que a revisão acertou e o que já tava no código

conferi cada ponto do PDF lendo o arquivo citado. entre parênteses vai onde olhei.

já tá no código

LIGAR REPO CHECA ACESSO
put_repos só aceita repo que o token do próprio usuário enxerga (repositories.rs)

WEBHOOK EM TRANSAÇÃO
delivery e processamento no mesmo BEGIN/COMMIT. se falhar, rollback e o reenvio do GitHub funciona. o frame 6 antigo desenhava errado, corrigi (webhooks.rs)

APAGAR MENSAGEM
remove() já faz content = '' (messages.rs). sobram autor, data e reações

MARKDOWN E LINKS
Link só aceita http, https e mailto, javascript: não passa (Markdown.tsx)

CSP
script-src 'self' e object-src 'none' já estão no tauri.conf.json

procede, fazer logo

COMANDOS DO TAURI SEM MANIFEST
capabilities só tem core:default e o build.rs não declara os comandos do app. um XSS chama start_share e transmite a tela da vítima

LOGIN TRAVA CONTA ALHEIA
a chave do bloqueio é só o username (auth.rs). como o nome aparece no chat, qualquer um trava qualquer um por 15 min. limitar por IP + conta, com atraso crescente

EXPULSAR NÃO DERRUBA A VOZ
achado meu: kick() só apaga o membro e publica member.left. a pessoa continua na sala do LiveKit

SEM APAGAR CONTA NEM EXPORTAR
não existe rota pra isso. é o buraco real de LGPD, mais que o soft delete

SEM BACKUP DO SQLITE
o banco é um arquivo só. Litestream resolve com pouco esforço

ROTAÇÃO DA GITHUB_TOKEN_KEY
crypto.rs guarda nonce + texto, sem versão da chave. trocar a chave invalida todos os tokens

PAYLOAD_JSON SEM RETENÇÃO
o detalhe de cada webhook fica pra sempre. definir prazo

só quando hospedar

MUTEX DO REFRESH TOKEN
o Mutex só vale com uma instância. com duas APIs precisa de lock no banco ou no Redis

INVALIDAR CACHE NA EXPULSÃO
hoje cada evento consulta role_of no banco, então o problema nem existe. aparece quando entrar o cache de membros

REDIS PODE PERDER EVENTO
o PDF diz que o resync cobre. só em parte: resync sai no Lagged e na reconexão do cliente, não quando a assinatura do Redis cai. a API precisa emitir resync nesse caso

SSRF NA PRÉVIA DE LINK
não existe prévia de link ainda. quando existir, bloquear IP privado

NOTIFICAÇÃO SEM CONTEÚDO
não existem notificações ainda

discordo em parte

OUTBOX TRANSACIONAL
prematuro. o webhook já comita antes de publicar e o cliente faz resync. só compensa quando existir feed de XP e insígnias de verdade, que não tá no repo

OBSERVABILIDADE
tracing e tracing-subscriber já estão nas dependências. falta log estruturado, métricas e relatório de erro no desktop

CRIPTOGRAFIA POR SERVIDOR
boa ideia, mas o PDF subestima o custo de KMS. ver frame 13

ordem que eu faria

1. manifest dos comandos do Tauri, login por IP + conta, kick derrubando a voz
2. rota de apagar e exportar conta, backup com Litestream, versão da chave, retenção do payload_json
3. interface MessageCipher e detecção de segredo (frame 13), antes de existir busca no servidor
4. o resto da arquitetura alvo, quando tiver usuário de verdade

proteção de dados sensíveis

ponta a ponta fica fora. o plano é cifrar na aplicação, com uma chave por servidor, e detectar segredo colado no chat.

4 camadas e onde o Squak está hoje

em trânsito (TLS)

- protege contra rede interceptada
- hoje: http e ws em localhost. TLS entra na hospedagem

em repouso (disco)

- protege contra roubo de disco ou backup
- hoje: arquivo SQLite sem cifra própria

na aplicação, chave por servidor

- protege contra vazamento do banco
- hoje: só os tokens do GitHub são cifrados;
- mensagens ficam em texto puro

ponta a ponta (E2EE)

- protege até contra o próprio Squak
- quebra busca, bots, GitHub no canal e histórico
- pra membro novo. só em DM, depois (MLS, RFC 9420)

fase 1: envelope encryption

KEK (chave mestra)

- começa no .env, igual a GITHUB_TOKEN_KEY
- KMS ou Vault só quando hospedar

cifra

cifra

DEK do servidor A

- cifrada pela KEK
- tabela server_keys

DEK do servidor B

- cifrada pela KEK
- tabela server_keys

cifra

cifra

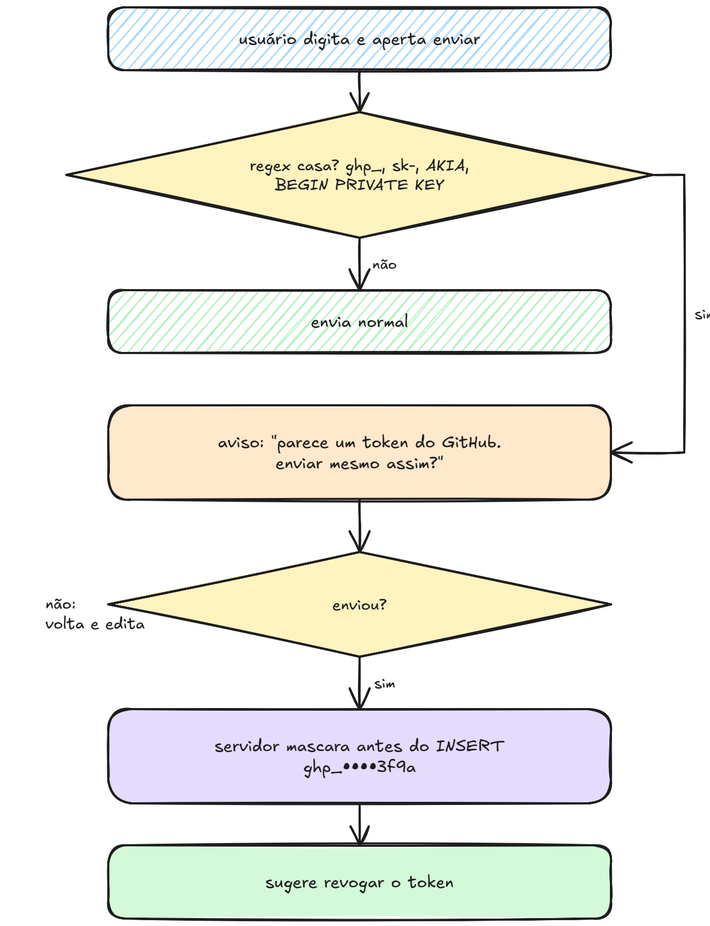
messages.content

- nonce + AES-256-GCM(texto)
- AAD = message_id + channel_id, então não dá pra copiar
- o texto cifrado de uma mensagem pra outra no banco

crypto-shredding: apagar a DEK do servidor torna todas as mensagens ilegíveis na hora, backups inclusos. serve pra excluir servidor e pra LGPD.

minha ressalva: a busca no servidor deixa de existir, e o frame 9 prevê full-text no Postgres. hoje não existe busca, então o custo agora é zero e só cresce depois.
por isso eu criaria a interface MessageCipher (um trait, como o GitHubApi)
já com a KEK no .env, e deixaria o KMS pra quando for hospedar.

detecção de segredo no Composer



decisão em aberto: mascarar só na tela deixa o segredo no banco, no backup e na notificação. ou guarda mascarado (revelar só pro autor), ou guarda o original cifrado à parte. regex no cliente também não garante nada, então a checagem precisa existir no servidor.

outros vazamentos, quando a feature existir

PRÉVIA DE LINK
bloquear IP privado, senão vira SSRF

NOTIFICAÇÃO DO SISTEMA
opção sem o texto da mensagem

LOGS
nunca registrar o conteúdo das mensagens

CACHE LOCAL
histórico no disco cifrado, chave no keyring

RETENÇÃO E AUDITORIA
apagar após X dias por servidor e log de ação de admin

o que dizer pros usuários: só o que for verdade. hoje as mensagens estão em texto puro no SQLite, então a frase "armazenadas cifradas com chave exclusiva do servidor" do PDF só vale depois da fase 1. e nunca escrever "ponta a ponta" nos canais.